REWARDING

Token Distribution Platform

Technical Whitepaper & Platform Reference

v1.0 — November 2025

Multi-project. Merkle-verified. Fully on-chain.

Table of Contents

§	Section	Page
1	Platform Overview	3
2	Architecture & Smart Contracts	4
3	Project Management	5
4	Campaign Lifecycle	6
5	Whitelist Implementation	8
6	Claim Verification & Merkle Proofs	10
7	Token Economics & Fee Structure	12
8	Security Model	13
9	Frontend Architecture	15
10	Deployment Guide	16
11	Supported Networks & Tokens	17
12	Glossary	18

© 2025 Rewarding Platform. For informational purposes only.

1. Platform Overview

Rewarding is a decentralised multi-project token distribution platform built on EVM-compatible blockchains. It allows any Web3 project to independently create token distribution campaigns, define eligible wallets, fund allocations and allow recipients to claim their tokens on-chain, without requiring a backend server, database or centralised infrastructure.

- No backend required — all logic lives in Solidity smart contracts.
- Multi-tenancy — unlimited projects and campaigns coexist on one contract deployment.
- Scalable whitelisting — Merkle tree proofs support millions of addresses with O(log n) verification.
- Direct registration — small campaigns (<100 addresses) can register allocations directly on-chain.
- Linear vesting — optional time-based unlock schedule for each campaign.
- Refundability — project owners reclaim unclaimed tokens after campaign expiry.
- Platform governance — admin role controls fee parameters and can pause malicious campaigns.

Actor	Description	Key permissions
Project Owner	Any wallet that creates a project profile	Create/fund campaigns, manage
Whitelisted User	Wallet in a campaign's allowlist	Claim allocated tokens during acti
Platform Admin	PLATFORM_ADMIN role holder	Set fees, pause campaigns/platfo
Claimant	Any wallet submitting a valid Merkle proof	Claim once per campaign in MER

© 2025 Rewarding Platform. For informational purposes only.

2. Architecture & Smart Contracts

The platform is implemented as a single Solidity contract **Rewarding.sol** inheriting OpenZeppelin's battle-tested modules. This minimises the attack surface and simplifies deployment.

Module	Purpose
ReentrancyGuard	Prevents reentrancy attacks on claim and refund functions
AccessControl	Role-based access: DEFAULT_ADMIN, PLATFORM_ADMIN, PROJ
Pausable	Emergency stop for entire platform or per-campaign pauses
IERC20 / SafeERC20	Safe token transfers with return-value checks
MerkleProof	On-chain Merkle proof verification (OpenZeppelin implementation)

Mapping	Key(s)	Value	Purpose
projects	projectId	Project struct	All registered project profiles
campaigns	campaignId	Campaign struct	All campaigns with status and allo
hasClaimed	campaignId, address	bool	Double-claim prevention
directAllocation	campaignId, address	uint256	Per-wallet allocation (DIRECT mo
claimedByUser	campaignId, address	uint256	Tokens already claimed (vesting s
projectCampaigns	projectId	uint256[]	All campaign IDs under a project

© 2025 Rewarding Platform. For informational purposes only.

3. Project Management

Any wallet can create one or more project profiles. A project is an on-chain record associating a wallet address with a name, branding assets and social links. It acts as the namespace under which campaigns are created and tracked.

```
function createProject(string name, string logoURI, string description, string website,
string twitter, string discord) returns (uint256 projectId)
```

On success, the caller is granted the PROJECT_OWNER role and a unique projectId is returned. The same wallet may create multiple projects. Project metadata can be updated at any time by the owner via updateProject(). The project record stores:

- id — unique sequential identifier
- owner — controlling wallet address
- name, logoURI, description — display metadata
- website, twitter, discord — social links
- active — soft-delete flag
- createdAt — block.timestamp at creation

© 2025 Rewarding Platform. For informational purposes only.

4. Campaign Lifecycle

A campaign is the atomic unit of token distribution. Each campaign belongs to exactly one project and has an independent token allocation, timing window, whitelist and status.

Status	Code	Transitions from	Description
ACTIVE	0	Creation / RESUME	Claims are open during the time window
PAUSED	1	ACTIVE	Claims blocked; owner or admin can pau
CLOSED	2	Any	Owner explicitly closes; refund becomes
EXPIRED	3	Implicit	Determined by block.timestamp > endTir

createCampaign() is payable. The project owner deposits the full token allocation plus the platform fee in a single transaction. For ERC-20 tokens, the owner must first approve the contract for totalAllocation + fee. For native tokens (BOT/ETH), msg.value covers both. No top-up after creation — the allocation is immutable.

Parameter	Type	Description
projectId	uint256	Owner's project (must be caller's project)
name	string	Display name shown in the UI
description	string	Campaign description
token	address	ERC-20 token address; address(0) for native token
totalAllocation	uint256	Total tokens to distribute (in wei / smallest unit)
startTime	uint256	Unix timestamp — must be >= block.timestamp
endTime	uint256	Unix timestamp — must be > startTime
merkleRoot	bytes32	Merkle root; bytes32(0) for DIRECT mode
whitelistMode	uint8	0 = MERKLE, 1 = DIRECT
allowPartialClaim	bool	Allow incremental vesting claims
vestingDuration	uint256	Seconds of linear vesting from startTime; 0 = instant

© 2025 Rewarding Platform. For informational purposes only.

Once a campaign ends (block.timestamp > endTime) or is explicitly closed by the owner, refundUnclaimed() may be called by anyone. The remaining balance (totalAllocation - claimedAmount) is returned to the project owner wallet. The function is guarded against reentrancy and sets claimedAmount = totalAllocation before transferring to prevent double-refunds.

© 2025 Rewarding Platform. For informational purposes only.

5. Whitelist Implementation

Rewarding supports two whitelist modes. The choice is made at campaign creation and cannot be changed after the first claim. Each mode has different on-chain gas costs and off-chain preparation requirements.

The campaign creator prepares an allowlist CSV containing wallet address and amount per line. A Merkle tree is constructed client-side using the inlined MerkleUtils JavaScript module shipped with the frontend. The Merkle root (a 32-byte hash) is registered on-chain via updateMerkleRoot(). Individual proofs are distributed to recipients off-chain (Discord, email, project website, or as a JSON file download).

```
leaf = keccak256(abi.encodePacked(address, uint256_amount))
```

The sortPairs=true configuration is used in both the JavaScript tree builder and must be assumed when re-constructing proofs externally. Each proof path costs approximately O(log2 N) on-chain keccak256 operations, making this viable for millions of addresses.

- Step 1: Prepare allowlist: address,amount CSV. Amount may be human-readable (100) or raw wei.
- Step 2: Upload CSV in the Whitelist Manager page — the browser builds the Merkle tree.
- Step 3: Copy the displayed Merkle root.
- Step 4: Register root on-chain via 'Register on Campaign' (calls updateMerkleRoot).
- Step 5: Download proof JSON — distribute to recipients via your preferred channel.
- Step 6: Recipients paste their proof into the Claim dashboard to verify eligibility and claim.

For small campaigns (<100 addresses), the project owner may call setDirectAllocations() to register wallet addresses and their allocations directly in the contract mapping. This eliminates proof distribution but costs gas per address registered (approximately 25,000 gas per entry on EVM chains).

```
function setDirectAllocations(uint256 campaignId, address[] wallets, uint256[] amounts)
```

Direct allocations can only be set before any claims are made (claimedAmount == 0). They cannot be updated incrementally — the entire batch must be re-submitted if corrections are needed. The total sum of direct allocations must not exceed the campaign's totalAllocation.

© 2025 Rewarding Platform. For informational purposes only.

6. Claim Verification & Merkle Proofs

A user submits a claim by calling claimMerkle(campaignId, amount, proof[]) where amount is the exact allocation in the Merkle leaf. The contract reconstructs the leaf and walks the proof path using MerkleProof.verify(). All of the following must hold for a claim to succeed:

- Campaign status must be ACTIVE.
- block.timestamp must be within [startTime, endTime].
- hasClaimed[campaignId][msg.sender] must be false (or claimedByUser < amount for vesting).
- MerkleProof.verify(proof, merkleRoot, leaf) must return true.
- Platform must not be paused (Pausable check).

If vestingDuration > 0, the claimable amount at any point is:

```
claimable = (fullAmount * elapsed) / vestingDuration — capped at fullAmount
```

where elapsed = block.timestamp - campaign.startTime. Users may call claimMerkle or claimDirect multiple times to harvest vested tokens incrementally, provided allowPartialClaim is true. The contract tracks the per-user claimed total in claimedByUser[campaignId][address] and only marks hasClaimed=true when the full allocation is exhausted.

Model	Mechanism	Privacy	Cost
Public JSON	GitHub release / website	Low	Free
Discord bot	DM on role verification	High	Bot hosting
Email	Sign-up gated distribution	Medium	Email provider
Claim link	Deep link with #campaign=ID	Medium	Free
On-chain event	Log indexer (The Graph)	Public	Graph node

```
{ "root": "0x...", "proofs": { "0xWallet": { "amount": "1000000000000000000000", "proof":
["0x...", "0x..."] } } }
```

© 2025 Rewarding Platform. For informational purposes only.

7. Token Economics & Fee Structure

Rewarding charges a platform fee on campaign creation. The fee is deducted from the total amount deposited and transferred immediately to the feeRecipient address. The allocation stored on the campaign is the net amount available to claimants.

Parameter	Value	Notes
Default fee	50 bps (0.5%)	Set at deployment
Maximum fee	500 bps (5%)	Hard-coded ceiling in contract
Fee recipient	Configurable	Set by PLATFORM_ADMIN at deploy or any tim
Fee timing	At creation	Transferred in the same tx as campaign funding
ERC-20 fee	Same token	Project approves allocation + fee before creating
Native fee	msg.value	msg.value must cover allocation + fee

No fee is charged on claims or refunds. The platform does not take custody of tokens beyond the escrow period — all tokens not claimed by campaign end are returned to the project owner via refundUnclaimed().

© 2025 Rewarding Platform. For informational purposes only.

8. Security Model

Threat	Mitigation
Reentrancy on claim/refund	ReentrancyGuard on all state-modifying external functions; state upda
Double claim	hasClaimed mapping checked before disbursement; set to true atomi
Invalid Merkle proof	MerkleProof.verify reverts on mismatch; leaf encodes both address a
Proof front-running	Leaf includes msg.sender — a copied proof is only valid for the origin
Campaign overfunding	totalAllocation is immutable after creation; contract holds exactly the
Malicious project	PLATFORM_ADMIN can pause individual campaigns or the entire pla
Fee manipulation	MAX_FEE_BPS ceiling of 500 bps enforced in setFee(); cannot be by
Token drain via refund	refundUnclaimed checks expiry/closure; sets claimedAmount = totalA
Access control bypass	OpenZeppelin AccessControl with explicit role checks; no tx.origin us
ERC-20 transfer failures	SafeERC20 wraps all transfers; reverts on false return value or missi
Native token loss	NativeTransferFailed custom error on failed .call{value}(); no silent fai
Vesting underflow	_vestingClaimable subtracts already-claimed amount with saturating

- Verify totalAllocation invariant: contract balance >= totalAllocation - claimedAmount for all campaigns.
- Confirm hasClaimed is set before _disburse in both claimMerkle and claimDirect.
- Test refundUnclaimed cannot be called twice on the same campaign.
- Fuzz Merkle proof verification with malformed proofs and boundary amounts.
- Confirm pauseCampaign by PLATFORM_ADMIN immediately blocks claims.
- Test createCampaign reverts when startTime < block.timestamp.
- Verify fee calculation rounds down (integer division), never over-charges.

© 2025 Rewarding Platform. For informational purposes only.

9. Frontend Architecture

The Rewarding frontend is a single HTML5/CSS3/Vanilla JavaScript file. It requires no build step, bundler or Node.js runtime — open index.html in any browser with MetaMask or a compatible wallet extension installed.

Component	Implementation	Why
Wallet connection	window.ethereum (EIP-1193)	Universal wallet standard
Contract calls	ethers.js v6 (CDN)	Mature ABI/signer abstraction
Merkle tree	Inlined MerkleUtils (pure JS)	No bundler; works offline
Styling	CSS custom properties + vanilla CSS	No framework dependency
State	In-memory JS object (STATE)	Ephemeral; re-fetched on connect
Persistence	None — read from chain	No backend, no database

Page	Route/ID	Description
Landing	landing	Platform hero, stats, featured campaigns
Campaign Explorer	explorer	Searchable, filterable list of all campaigns
Project Directory	projects	All registered projects with metadata
Campaign Detail	campaign-detail	Full campaign info + user eligibility + claim butto
Create Project	create-project	On-chain project creation form
Create Campaign	create-campaign	Campaign creation + token approval flow
Whitelist Manager	whitelist	CSV upload → Merkle root → proof download
Claim Dashboard	claim	User's eligibility + claim history
Owner Dashboard	dashboard	Campaign management for project owners
Platform Admin	admin	Fee config, campaign control (admin only)

© 2025 Rewarding Platform. For informational purposes only.

10. Deployment Guide

npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox @openzeppelin/contracts merkletreejs keccak256 ethers

Add network entries for botchain, celo or your target. Set the RPC URL and deployer private key via environment variables.

export FEE_RECIPIENT=0xYourFeeWallet && export FEE_BPS=50

npx hardhat test

npx hardhat run scripts/deploy.js --network botchain_testnet

The deploy script calls verify:verify automatically. For manual: npx hardhat verify --network botchain

Replace CONTRACT_ADDRESS in index.html with the deployed address. The ABI is already embedded.

Create a project → create a campaign → upload a 3-address CSV → register root → claim from a whitelisted wallet.

© 2025 Rewarding Platform. For informational purposes only.

11. Supported Networks & Tokens

Rewarding deploys on any EVM-compatible chain. The table below lists known configurations. **Always verify token contract addresses on the official chain explorer before use.** Rewarding does not hardcode token addresses — they are supplied by the campaign creator.

Network	Chain ID	Native token	USDT address (verify before use)
BOT Chain	1991	BOT	Verify on bot-scan.io — do not guess
Celo mainnet	42220	CELO	0x617f3112bf5397D0467D315cC709EF968D9b
Celo Alfajores	44787	CELO	Testnet faucet token — varies
Ethereum	1	ETH	0xdAC17F958D2ee523a2206206994597C13D8
Polygon	137	MATIC	0xc2132D05D31c914a87C6611C10748AEb04E
Hardhat local	31337	ETH	Deploy MockERC20 from tests/

■ **BOT Chain USDT: The official USDT contract address on BOT Chain must be verified independently through the official BOT Chain block explorer (bot-scan.io) or the official BOT Chain documentation before being used in a campaign. Never trust an address from an unofficial source.**

© 2025 Rewarding Platform. For informational purposes only.

12. Glossary

Term	Definition
Allocation	The total amount of tokens set aside for a campaign, deposited at creation.
Basis points (bps)	1/100th of a percent. 100 bps = 1%. Used for fee configuration.
Campaign	An atomic token distribution event under a project, with its own token, timing, whitelist and status.
Claim	The act of a whitelisted wallet withdrawing their allocated tokens from the contract.
DIRECT mode	Whitelist mode where wallet-to-amount mappings are stored on-chain in the contract.
Escrow	The contract holds campaign funds in trust until claimed or refunded.
Funder	The wallet that created and funded a campaign (always the project owner).
Leaf	An entry in the Merkle tree: keccak256(address, amount).
MERKLE mode	Whitelist mode where only the Merkle root is stored on-chain; proofs are off-chain.
Merkle proof	A series of sibling hashes allowing reconstruction of the Merkle root from a leaf.
Merkle root	The 32-byte root hash of the Merkle tree, stored on-chain per campaign.
Platform admin	Wallet holding the PLATFORM_ADMIN role; controls fee and emergency pause.
Project	An on-chain namespace associating a wallet with metadata and campaigns.
Proof distribution	The mechanism by which Merkle proofs are delivered to whitelisted wallets.
SafeERC20	OpenZeppelin wrapper that checks ERC-20 transfer return values.
sortPairs	Merkle tree convention: sibling hashes are sorted before combining, ensuring deterministic roots.

© 2025 Rewarding Platform. For informational purposes only.

Term	Definition
Vesting	Linear time-based unlock: a fraction of the allocation becomes claimable per second.
Wei	The smallest denomination of ERC-20 tokens; 1 token = 10^decimals wei.

End of Rewarding Platform Technical Whitepaper v1.0

Built for the Web3 ecosystem. Audits strongly recommended before mainnet deployment.

© 2025 Rewarding Platform. For informational purposes only.